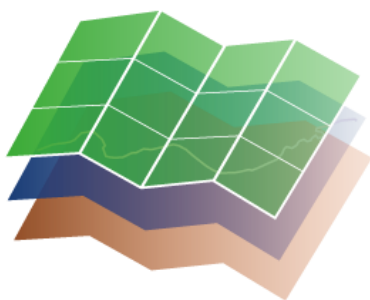




MAPA

CIUDADANO

Lanzamiento de procesos en sistema operativo Windows

11/07/2025
Versión 1.1

Contenido

Con contenedor de entornos Python - Anaconda	4
Crear entorno virtual	4
Proceso primero: 1Proceso-PostGISToGeojson	5
Proceso segundo: 2Proceso-ValidacionGeojson	8
Resumen de ficheros para la configuración del lanzamiento en sistema operativo Windows	11
Listado de ficheros de configuración y scripts para su parametrización	11
Listado de scripts con modificaciones para el cambio de sistema operativo	11

Versiones

Versión	Fecha	Comentarios
1.0	23/01/2025	Creación y actualización
1.1	11/07/2025	Se incluyen las validaciones geométricas

Con contenedor de entornos Python - Anaconda



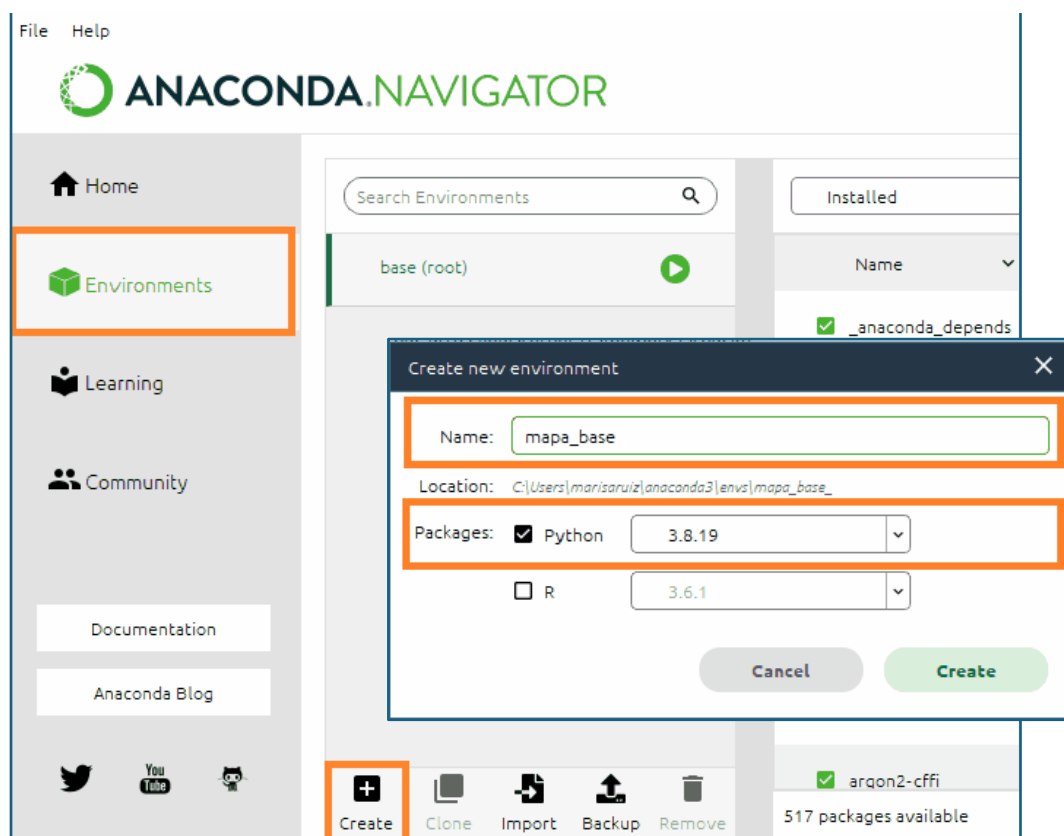
<https://www.anaconda.com/>

Anaconda es una distribución gratuita y de código abierto del lenguaje de programación Python y R, que se utiliza principalmente en ciencia de datos y aprendizaje automático. Facilita la instalación, gestión y despliegue de entornos de desarrollo, permitiendo a los usuarios crear y manejar entornos aislados con diferentes versiones de Python y paquetes específicos para sus proyectos.

Crear entorno virtual

Una vez instalado Anaconda se crea el entorno virtual en el que vamos a trabajar.

- *Environments/Create* y se asigna un nombre y lenguaje. Ten en cuenta que las librerías que hay que instalar más tarde te van a requerir una versión de Python >3.6 <3.10, nosotros te recomendamos que instales **3.8.19**, todo el proceso está probado para windows con esa versión.



Una vez que el entorno de trabajo está creado, aparece en la lista de entornos y si pulsamos *play* abriremos su terminal.

Desde el terminal podremos instalar los paquetes python que necesitamos para las tareas de Mapa Base:

- La instalación de las librerías externas necesarias para la ejecución de los procesos (`html-to-json`, `numpy`, `psycopg2`, `pyexcel-ods3`), se puede realizar invocando su instalación una a una:
 - `pip3 install "nombre de la librería"`
- U otra opción es instalar directamente las librerías desde el archivo `scripts/mapeo/1Proceso-PostGISToGeojson/requirements.txt`:
 - `pip3 install -r requirements.txt`
 - Esta opción requiere que informes la ruta donde se encuentra el fichero `requirements.txt`
 - Ejemplo:
 - `pip3 install -r C:\Proyectos\mapabase-gh-pages\scripts\mapeo\1Proceso-PostGISToGeojson\requirements.txt`
 - Instalar el módulo `joblib`, te hará falta en el proceso 2 (validación):
 - `pip3 install joblib`
 - Instalar `geopandas`, hará falta para las validaciones geométricas
 - `pip3 install geopandas`

Una vez que todo está correctamente instalado se podrían ejecutar todos los scripts del proceso [`python nombredelscript.py`] pero es necesario parametrizarlos correctamente. Se describe paso a paso a continuación.

Proceso primero: 1Proceso-PostGISToGeojson

Este proceso extrae la información de una base de datos postgis y la transforma a formato geojson siguiendo las pautas marcadas en un fichero `.ods`. Este fichero es una hoja de cálculo en la que, entre otras cuestiones, se indica qué datos se van a migrar a geojson y por tanto van a formar parte finalmente de las teselas vectoriales.

Tienes información detallada sobre ella en:

<https://github.com/IDEESpain/mapabase/tree/gh-pages/scripts/mapeo/1Proceso-PostGISToGeojson>:

- Datos de entrada proceso `.ods` Proveedores de datos

Proceso 1. Script 1: `run_postGISToGeojson.py`

Para lanzar el primer script del proceso 1 y extraer de la base de datos postGIS la información conforme al modelo de datos de Mapa Base necesitaremos hacer modificaciones en los scripts proporcionados (parametrización). Para que estas modificaciones sean las mínimas posibles lanzaremos el intérprete de python desde la carpeta `mapabase_gh_pages` que es donde nos hemos descargado el repositorio de github (cd.. subes un nivel en la ruta, cd entras en el directorio que indiques de esa carpeta), ya que los scripts están preparados para trabajar con rutas relativas.

```
C:\WINDOWS\system32\cmd.exe - python C:\Proyectos\mapabase-gh-pages\scripts\mapeo\1Proceso-PostGISToGeojson\run_postGISToGeojson.py
(mapa_base_2) C:\Users>cd..
(mapa_base_2) C:\>cd Proyectos
(mapa_base_2) C:\Proyectos>cd mapabase-gh-pages
```

Ten en cuenta, también, que:

- La hoja de cálculo debe estar bien configurada, si no dará error o no obtendremos los resultados esperados.

El fichero `conex.json` [\\mapabase-gh-pages\scripts\mapeo\1Proceso-PostGISToGeojson\lib\conex.json] debe contener los datos de conexión a las base de datos que vayas a referenciar en la hoja de cálculo.

```
{
  "NOMBRE_CONEXION": { "ip": "XXX.XXX.XXX.XXX", "puerto": "XXXX",
    "usuario": "nombre_usuario", "contrasena": "password", "bbdd":
    "nombre_base_de_datos", "esquema": "nombre_esquema" }
}
```

- Para lanzar los script con sistema operativo Windows deberemos modifica el script `mapeo.py`

..\mapabase-gh-pages\scripts\mapeo\1Proceso-PostGISToGeojson\lib\mapeo.py

```
147 def procesoMapeo(self):
148     #Crea log del proceso
149     start_time_total = time.time()
150     ahora = datetime.now()
151     anno = ahora.strftime("%Y")
152     mes = ahora.strftime("%m")
153     dia = ahora.strftime("%d")
154     hora = ahora.strftime("%H")
155     min = ahora.strftime("%M")
156     seg = ahora.strftime("%S")
157     subNombre = anno+mes+dia+' '+hora+min+seg
158     ##### Para Windows descomenta la siguiente línea (línea 159) y comenta la línea 161
159     archivoLOG = codecs.open(self.path_carpetaSalida+'\\'+subNombre+' ToGeojson '+self.proveedor+'.log', "a", "utf-8")
160     ##### Para Linux descomenta la siguiente línea (línea 161) y comenta la inmediata anterior (línea 159)
161     # archivoLOG = codecs.open(self.path_carpetaSalida+'/' +subNombre+' ToGeojson '+self.proveedor+'.log', "a", "utf-8")
162     archivoLOG.write(" ### --- Log del proceso de mapeo y transformación de PG a Geojson --- ### \n")
163     archivoLOG.write(" # ----- \n")
164     archivoLOG.write(" \n")
165     archivoLOG.write(" # Inicio: {} \n".format( ahora.strftime("%m/%d/%Y, %H:%M:%S")) )
166     archivoLOG.write(" \n")
167     archivoLOG.close()
```

Además, el script de Python [`run_postGISToGeojson.py`] debe estar parametrizado correctamente para que localice esta hoja de cálculo, la carpeta de salida donde se alojarán los ficheros geojson que se generen (debes crearla primero) y el fichero `conex.json` que recoge los datos de conexión a la base de datos postgis.

```
import time
from lib.mapeo import mapeo_PG_GJSON

if __name__ == '__main__':
    # # # parámetros del script
    path_jsonConex = 'scripts\\mapeo\\1Proceso-PostGISToGeojson\\lib\\conex.json'
    path_hojaCalculoMApeo = 'scripts\\mapeo\\1Proceso-PostGISToGeojson\\CNIG_ToGeojson_202403111206.ods'
    path_carpetaSalida = 'scripts\\mapeo\\1Proceso-PostGISToGeojson\\salidaGeojson_202403111206'
    proveedor = 'cnig'
```

Detalle de `run_postGISToGeojson.py`: parámetros de configuración del script

En este caso, hemos creado una carpeta para la almacenar los ficheros geojson de salida del script marcada con la misma fecha del fichero `.ods` de transformación:

- ..\\salidaGeojson_202403111206

Esto nos permite controlar qué salida se corresponde con qué fichero `.ods` en el caso de realizar exportaciones en momentos diferentes.

Dentro de la carpeta se almacenan los ficheros de salida junto con el fichero `.log` que informa del procesamiento realizado.

Para lanzar el script, en la ventana de la terminal, ejecutamos:

- `python scripts\mapeo\1Proceso-PostGISToGeojson\run_postGISToGeojson.py`

```
### --- Log del proceso de mapeo y transformación de PG a Geojson --- ###
#
# Inicio: 07/01/2024, 05:31:55
# Órdenes de conexión únicas: 2
# -----

##### construccion_lin #####
1º Orden de conexión: MAPABASE construccion_lin construccion_lin

Sentencia SQL: SELECT ST_AsGeoJSON( ST_SnapToGrid ( ST_Force2D( ST_Transform(the_geom,4258) ), 0.0000001) ), clase, proveedor FROM "public"."construccion_lin"
Número de entidades: 3760729

listado de mapeos realizados:
clase: auxiliar_construido --> registros: 384879
proveedor: * --> registros: 3760729
clase: auxiliar_construido_vial --> registros: 86571
clase: cercado --> registros: 1323055
clase: compuerta --> registros: 302
clase: escollera --> registros: 590
clase: muelle --> registros: 999
clase: muralla --> registros: 1073
clase: muro --> registros: 1745177
clase: muro_contención --> registros: 91718
clase: pantalan --> registros: 1811
clase: pasarela --> registros: 3492
clase: presa --> registros: 100537
clase: puente --> registros: 10851
clase: túnel --> registros: 9646
clase: transporte_suspendido --> registros: 28

Entidades no procesadas: 0
tiempo de ejecución: 1949.9618980884552 seg
# -----

##### construccion_pol #####
2º Orden de conexión: MAPABASE construccion_pol construccion_pol

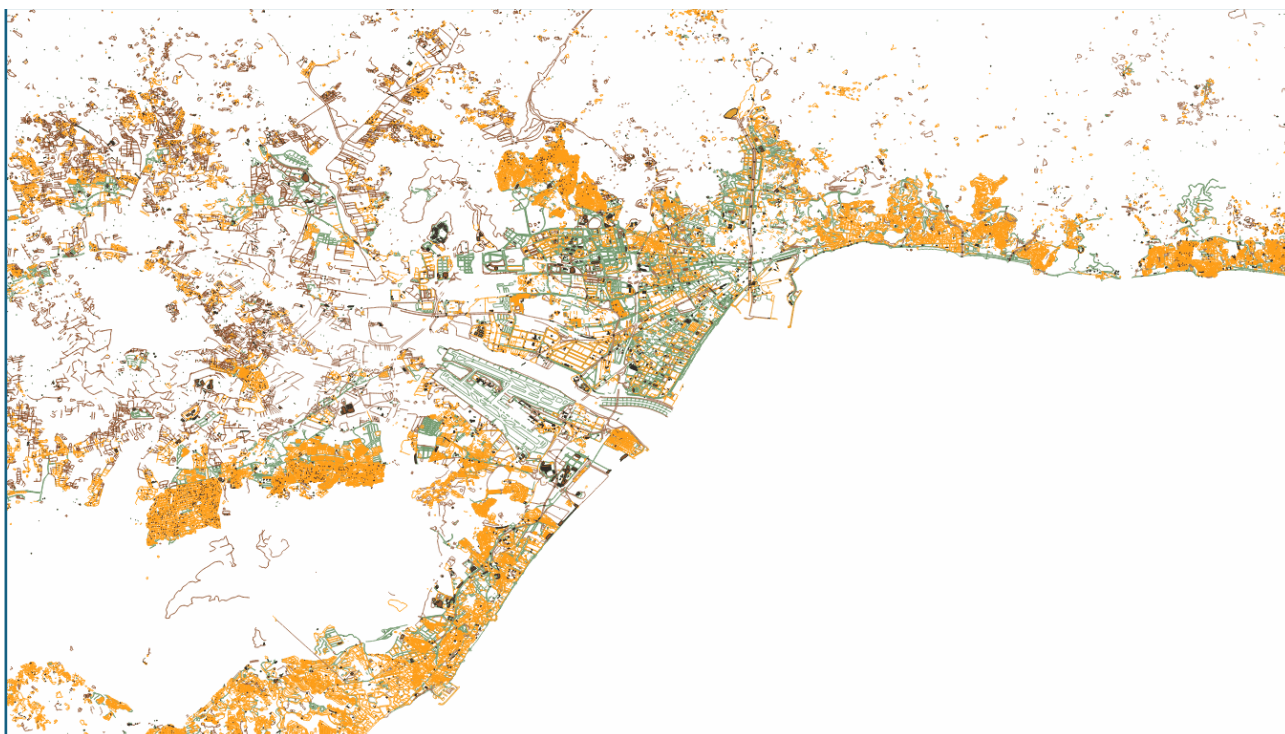
Sentencia SQL: SELECT ST_AsGeoJSON( ST_SnapToGrid ( ST_Force2D( ST_Transform(the_geom,4258) ), 0.0000001) ), clase, proveedor FROM "public"."construccion_pol"
```

Detalle del fichero .log de exportación

Si todo ha ido bien todas las entidades se habrán procesado:

- Entidades no procesadas: 0

Y podrás ver la información en Geojson en un SIG, por ejemplo QGIS:



Detalle de las capas construccion_lin y construccion_pol tras su exportación

Proceso segundo: 2Proceso-ValidacionGeojson

Puedes leer la documentación disponible en el proyecto:

- <https://github.com/IDEESpain/mapabase/tree/gh-pages/scripts/mapeo/2Proceso-ValidacionGeojson>

El primer paso antes de lanzar los scripts de validación es parametrizar correctamente el fichero de configuración **config.json**. En windows debes incluir dos barras (\\) para separar los directorios.

```
1 {
2     "geojson_folder_path": "scripts\\mapeo\\1Proceso-PostGISToGeojson\\salidaGeojson_20250612\\",
3     "fgb_folder_path": "scripts\\mapeo\\1Proceso-PostGISToGeojson\\salidaGeojson_20250612\\fgb\\",
4     "git_carpeta_mapabase_gh_pages" : "C:\\Proyectos\\mapabase-gh-pages\\",
5     "JSON_comprobacion" : "scripts\\mapeo\\2Proceso-ValidacionGeojson\\lib\\comprobacion.json",
6     "JSON_config_Tipecanoe" : "scripts\\teselas\\4proceso-vtilesToMbTiles\\config_vtiles.json",
7     "fgb_temp_folder" : "scripts\\mapeo\\1Proceso-PostGISToGeojson\\salidaGeojson_20250612\\fgb_temp\\",
8     "geometrias_corregir" : "scripts\\mapeo\\1Proceso-PostGISToGeojson\\salidaGeojson_20250612\\Corregir_geom\\",
9     "json_corregir_geometrias" : "scripts\\mapeo\\1Proceso-PostGISToGeojson\\salidaGeojson_20250612\\corregir_geometrias.log"
10 }
```

config.json proceso 2

Las carpetas de los fgb (fgb y fgb_temp) las deberías crear para la transformación a fgb pero queda fuera del alcance de este documento en el que llegaremos hasta validar los geojson para confirmar que son correctos antes de enviarlos.

Proceso 2. Script 1: run_crearJSONValidacion.py

El siguiente paso es crear el fichero el json de validación:

- `python scripts\\mapeo\\2Proceso-ValidacionGeojson\\run_crearJSONValidacion.py`

Para que corra en Windows tendrás que hacer cambios ya que la forma de almacenar las direcciones relativas cambian en uno y otro sistema operativo.

- En el script `run_crearJSONValidacion.py`

```
1
2 import time
3 import json
4 from lib.controlCalidad_GIT import control_calidad_GJSON
5
6 ### Para Windows
7 with open ("scripts\\mapeo\\2Proceso-ValidacionGeojson\\config.json") as f:
8     var_dict=json.load(f)
9
10 ### Para Linux
11 #with open (".\\config.json") as f:
12 #    var_dict=json.load(f)
```

- En el script `lib\\controlCalidad_GIT.py`

```
1 from logging import raiseExceptions
2 import os, time, sys, json, codecs, gc
3 from datetime import datetime
4 import html_to_json
5
6 from xml.etree import ElementTree as ET
7
8
9
10 class control_calidad_GJSON:
11
12     def __init__(self, path_carpetaEntrada):
13         self.path_carpetaEntrada = path_carpetaEntrada
14
15     ### Para Windows
16     with open ("C:\\Proyectos\\mapabase-gh-pages\\scripts\\mapeo\\2Proceso-ValidacionGeojson\\config.json") as f:
17         var_dict=json.load(f)
18
19     ### Para Linux
20     # with open (".\\config.json") as f:
21     #     var_dict=json.load(f)
22
23     self.verbose = False
24     self.git_carpeta_mapabase_gh_pages = var_dict["git_carpeta_mapabase_gh_pages"]
25     self.JSON_comprobacion = var_dict["JSON_comprobacion"]
26     self.JSON_config_Tipecanoe = var_dict["JSON_config_Tipecanoe"]
```

Cuando se configuran correctamente los scripts y se lanza, se crea desde la web (github mapabase) a partir del html. Si hay cambios en el modelo se tiene que actualizar el repositorio en local y volver a

lanzar. Si no ha habido cambios en el modelo se puede utilizar el archivo (/lib/comprobacion.json) anterior.

Más detalles en: <https://github.com/IDEESpain/mapabase/tree/gh-pages/scripts/mapeo/2Proceso-ValidacionGeojson>

```
C:\WINDOWS\system32\cmd.exe
(mapa_base_2) C:\Proyectos\mapabase-gh-pages\scripts\mapeo\2Proceso-ValidacionGeojson>python run_crearJSONValidacion.py
aerodromo_pol {'td': [{'code': {'_value': 'clase'}}, {'_value': 'string'}, {}, {'_value': 'Clase de objeto'}]}
aerodromo_pol {'td': [{'code': {'_value': 'estado'}}, {'_value': 'string'}, {}, {'_value': 'Situación en que se encuentra el objeto geográfico'}]}
aerodromo_pol {'td': [{'code': {'_value': 'nombre'}}, {'_value': 'string'}, {'_value': '[nombre]'}, {'_value': 'Nombre'}]}
aerodromo_pol {'td': [{'code': {'_value': 'proveedor'}}, {'_value': 'string'}, {'_value': '[codigo_proveedor]'}, {'_value': 'Proveedor de la información (lista controlada)'}]}
aerodromo_pol {'td': [{'code': {'_value': 'ref'}}, {'_value': 'string'}, {'_value': '[designador]'}, {'_value': 'Referencia o nombre abreviado. Si no se conoce o no es aplicable dejarlo en blanco'}]}
aerodromo_pto {'td': [{'code': {'_value': 'clase'}}, {'_value': 'string'}, {}, {'_value': 'Clase de objeto'}]}
aerodromo_pto {'td': [{'code': {'_value': 'nombre'}}, {'_value': 'string'}, {'_value': '[nombre]'}, {'_value': 'Nombre'}]}
aerodromo_pto {'td': [{'code': {'_value': 'proveedor'}}, {'_value': 'string'}, {'_value': '[codigo_proveedor]'}, {'_value': 'Proveedor de la información (lista controlada)'}]}
aerodromo_pto {'td': [{'code': {'_value': 'ref'}}, {'_value': 'string'}, {'_value': '[designador]'}, {'_value': 'Referencia o nombre abreviado. Si no se conoce o no es aplicable dejarlo en blanco'}]}
aerodromo_pto {'td': [{'code': {'_value': 'tipo'}}, {'_value': 'string'}, {}, {'_value': 'Categoría'}]}
aerodromo_pto {'td': [{'code': {'_value': 'uso'}}, {'_value': 'string'}, {}, {'_value': 'Tipo de utilización del objeto geográfico'}]}
aerodromo_pto {'td': [{'code': {'_value': 'estado'}}, {'_value': 'string'}, {}, {'_value': 'Situación en que se encuentra el objeto geográfico'}]}
altimetria_lin {'td': [{'code': {'_value': 'altitud'}}, {'_value': 'int'}, {'_value': '[altitud]'}, {'_value': 'Altitud'}]}
altimetria_lin {'td': [{'code': {'_value': 'clase'}}, {'_value': 'string'}, {}, {'_value': 'Clase de objeto'}]}
altimetria_lin {'td': [{'code': {'_value': 'proveedor'}}, {'_value': 'string'}, {'_value': '[codigo_proveedor]'}, {'_value': 'Proveedor de la información (lista controlada)'}]}
altimetria_pto {'td': [{'code': {'_value': 'altitud'}}, {'_value': 'int'}, {'_value': '[altitud]'}, {'_value': 'Altitud'}]}
altimetria_pto {'td': [{'code': {'_value': 'clase'}}, {'_value': 'string'}, {}, {'_value': 'Clase de objeto, tipo de punto altimétrico'}]}
altimetria_pto {'td': [{'code': {'_value': 'proveedor'}}, {'_value': 'string'}, {'_value': '[codigo_proveedor]'}, {'_value': 'Proveedor de la información (lista controlada)'}]}
aparcamiento_pol {'td': [{'code': {'_value': 'clase'}}, {'_value': 'string'}, {}, {'_value': 'Clase de objeto'}]}
aparcamiento_pol {'td': [{'code': {'_value': 'nombre'}}, {'_value': 'string'}, {'_value': '[nombre]'}, {'_value': 'Nombre'}]}
aparcamiento_pol {'td': [{'code': {'_value': 'proveedor'}}, {'_value': 'string'}, {'_value': '[codigo_proveedor]'}, {'_value': 'Proveedor de la información (lista controlada)'}]}
aparcamiento_pto {'td': [{'code': {'_value': 'clase'}}, {'_value': 'string'}, {}, {'_value': 'Clase de objeto'}]}
aparcamiento_pto {'td': [{'code': {'_value': 'nombre'}}, {'_value': 'string'}, {'_value': '[nombre]'}, {'_value': 'Nombre'}]}
aparcamiento_pto {'td': [{'code': {'_value': 'proveedor'}}, {'_value': 'string'}, {'_value': '[codigo_proveedor]'}, {'_value': 'Proveedor de la información (lista controlada)'}]}
area_servicio_pol {'td': [{'code': {'_value': 'clase'}}, {'_value': 'string'}, {}, {'_value': 'Clase de objeto'}]}
area_servicio_pol {'td': [{'code': {'_value': 'nombre'}}, {'_value': 'string'}, {'_value': '[nombre]'}, {'_value': 'Nombre'}]}
area_servicio_pol {'td': [{'code': {'_value': 'proveedor'}}, {'_value': 'string'}, {'_value': '[codigo_proveedor]'}, {'_value': 'Proveedor de la información (lista controlada)'}]}
area_servicio_pto {'td': [{'code': {'_value': 'clase'}}, {'_value': 'string'}, {}, {'_value': 'Clase de objeto'}]}
area_servicio_pto {'td': [{'code': {'_value': 'nombre'}}, {'_value': 'string'}, {'_value': '[nombre]'}, {'_value': 'Nombre'}]}
area_servicio_pto {'td': [{'code': {'_value': 'proveedor'}}, {'_value': 'string'}, {'_value': '[codigo_proveedor]'}, {'_value': 'Proveedor de la información (lista controlada)'}]}
autopista_lin {'td': [{'code': {'_value': 'clase'}}, {'_value': 'string'}, {}, {'_value': 'Clase de objeto'}]}
```

Detalle del proceso de creación del fichero comprobacion.json

Proceso 2. Script 2: run_validationProcess.py

Una vez que disponemos del archivo de validación, antes de lanzar el proceso, tendremos que modificar el script `run_validationProcess.py` para que corra en Windows.

```
1 import json
2 import time
3 from lib.controlCalidad_GIT import control_calidad_GJSON
4
5 ### Para Windows
6 #with open ("scripts\\mapeo\\2Proceso-ValidacionGeojson\\config.json") as f:
7 #    var_dict=json.load(f)
8
9 ### Para Linux
10 with open ("config.json") as f:
11     var_dict=json.load(f)
12
13 ### parámetros del programa
14 path_carpetasalidaGJSON = var_dict["geojson_folder_path"]
15
16 ### Se llama a la clase con los parámetros de la ejecución
17 ProcesoControlCalidad = control_calidad_GJSON(path_carpetasalidaGJSON)
18 ProcesoControlCalidad.verbose = True
19
20 ### Pasar de repositorio e GIT a JSON
21 # ProcesoControlCalidad.elementosAJSON()
22
23
24 ### Lanzar proceso en Windows
25 #start_time = time.time()
26 #ProcesoControlCalidad.JSON_comprobacion = 'scripts\\mapeo\\2Proceso-ValidacionGeojson\\lib\\comprobacion.json'
27 #ProcesoControlCalidad.procesoControlCalidad()
28
29 ### Lanzar proceso en Linux
30 start_time = time.time()
31 ProcesoControlCalidad.JSON_comprobacion = 'lib\\comprobacion.json'
32 ProcesoControlCalidad.procesoControlCalidad()
33
34 print("--- {} min de ejecución ---".format( (time.time() - start_time) /60 ) )
35
```

Una vez configurado el archivo lanzamos el script:

- `python scripts\mapeo\2Proceso-ValidacionGeojson\run_validationProcess.py`

Proceso 2. Script 3: run_corregir_geometrias.py

Para corregir las geometrías de los ficheros generados pasamos estos a la carpeta que hemos creado "corregir_geom" (geojson y vrt) y que hemos referenciado en el `config.json` y configuramos el script `run_corregir_geometrias.py` para que pueda correr en Windows:

```

1  import os
2  import json
3  import geopandas as gpd
4  from shapely.validation import make_valid
5  import gc
6  import re
7
8  # Cargar configuración desde config.json
9  ### Para Windows
10 #with open ("scripts\mapeo\2Proceso-ValidacionGeojson\config.json") as f:
11 #    var_dict=json.load(f)
12
13 ### Para Linux
14 with open ("config.json") as f:
15     var_dict=json.load(f)
16
17 # Ruta de la carpeta con los archivos geoespaciales (definida en config.json)
18 input_folder = var_dict["geometrias_corregir"]
19
20 # Archivo de salida para el listado de archivos corregidos (definido en config.json)
21 output_json = var_dict["json_corregir_geometrias"]
22
23 def correct_invalid_geometries(file_path, corrected_files):
24     """
25     Corrige las geometrías inválidas en un archivo FGB o GeoJSON.
26     Retorna True si se realizaron correcciones, False si no se necesitó.
27     """

```

Una vez que el script está modificamos lo podemos ejecutar:

- `python scripts\mapeo\2Proceso-ValidacionGeojson\run_corregir_geometrias.py`

Resumen de ficheros para la configuración del lanzamiento en sistema operativo Windows

Listado de ficheros de configuración y scripts para su parametrización

Proceso	script	línea(s)
1Proceso-PostGISToGeojson	\lib\ conex.json	Todas
1Proceso-PostGISToGeojson	\run_postGISToGeojson.py	6, 7, 8, 9
2Proceso-ValidacionGeojson	\config.json	Todas

Listado de scripts con modificaciones para el cambio de sistema operativo

Proceso	script	Línea para Windows	Línea para Linux
1Proceso-PostGISToGeojson	\\lib\mapeo.py	159	161
2Proceso-ValidacionGeojson	run_crearJSONValidacion.py	7-8	11-12
2Proceso-ValidacionGeojson	\\lib\controlCalidad_GIT.py	16-17	20-21
2Proceso-ValidacionGeojson	run_validationProcess.py	6-7	10-11
		25-27	30-32
2Proceso-ValidacionGeojson	run_corregir_geometrias.py	10-11	14-15